

When Do Agent Scaffolds Actually Help? Toward Compute-Normalized Evaluation of Workflow-Guided LLM Agents

Srijan Choudhary

Error606 Labs · srijan@inculcate.in

Status: preprint. This measurement-agenda and pilot-harness paper is not a fully powered empirical benchmark.

ABSTRACT

We develop a measurement agenda for testing when workflow scaffolds improve LLM-agent reliability under matched compute budgets. Agent systems increasingly add planning, verification, memory, tool use, reflection, role separation, and human gates, but many evaluations conflate scaffold value with extra inference, extra samples, or benchmark-specific tool ergonomics. We propose a regime-based protocol that compares unstructured agents, repeated unstructured agents, plan-first agents, and plan-plus-check workflows across tasks annotated by blocker type, verifier reliability, and harness/tool surface. An 18-task deterministic smoke harness validates the instrumentation, and an initial Hermes-delegate canary on five toy tasks illustrates the intended analysis: unstructured direct answers solved 5/5 tasks, plan-first and plan-plus-check solved 4/5, and best-of-2 matched direct success with twice the model calls; the two planned conditions failed a JSON-only task by adding prose before the object. These pilots are call-accounted but not token- or dollar-matched, and they are not evidence about general LLM agents. They instead expose the central measurement problem: scaffolds should be reported as marginal reliability per model call, tool call, token, dollar, and failure regime, not as raw success improvements alone.

Keywords: LLM agents · workflow scaffolds · compute-normalized evaluation · Pareto frontier · agent reliability · verifier regimes

1 Introduction

LLM agents are rarely deployed as bare model calls. They are wrapped in workflow scaffolds: reasoning-action loops, plans, tool calls, memory, reflection, verification, role separation, and sometimes humans in the loop. ReAct established the basic pattern of interleaving reasoning and environment actions [1]; Tree of Thoughts and LATS-style methods turn reasoning into search; Reflexion and Self-Refine add iterative feedback; workflow-guided systems such as FlowBench and Agent Workflow Memory provide or learn reusable procedures [2, 3]. In software engineering, SWE-agent emphasizes the agent-computer interface [4], while Agentless shows that a low-autonomy, high-structure localize-repair-validate workflow can be competitive without open-ended autonomous action selection [5].

The field's implicit bet is that more scaffold structure buys more reliability. That bet is plausible but under-specified. A plan-and-verify agent may outperform a one-shot prompt because the verifier is useful, because the plan decomposes the task, because the method spends more model calls, because it samples more candidates, or because the harness delivers tool outputs in a friendlier format. Cost-aware evaluation work argues that agent benchmarks must jointly report accuracy and cost, compare to simple baselines, and identify Pareto-dominated systems [6]. Recent scaffold-interaction work also cautions that adding all components can make a system worse than a smaller subset [7].

This paper asks a narrower question: when does a realized workflow graph buy reliability per unit compute? We treat scaffold value as regime-specific. A workflow should help most when a task has meaningful decomposition, observable intermediate state, and verifier feedback that is cheaper or more reliable than additional unconstrained generation. It should help less, or hurt, when the task is already easy, the verifier is brittle, the output format is strict, or the scaffold adds failure surface without new information.

Our contributions are:

1. A taxonomy that separates autonomy from workflow structure and treats planning, verification, memory, tool use, human gates, and role separation as measurable scaffold components.
2. A protocol for compute-normalized evaluation that includes matched-budget unstructured sampling baselines, success per model call, success per tool call, success per dollar, and Pareto-frontier labels; the present pilots implement call accounting but not full token/dollar matching.
3. A regime-map schema that logs task family, blocker type, verifier reliability and calibration, retrieval mode, tool-delivery path, harness surface, execution-control/workflow-executor policy, runtime guardrail/dataflow policy, and safety/state-mutation profile.
4. A working pilot harness with deterministic smoke tests and an initial Hermes-delegate canary showing how scaffold overhead can be dominated on easy tasks and how planning prose can break strict JSON verifiers.

2 Definitions and Taxonomy

We define an agent scaffold as any structure outside the base model call that constrains, sequences, evaluates, stores, or routes model behavior. This includes prompt-level structures such as plans, runtime structures such as verifier loops, memory structures such as induced workflows, and system-level structures such as role-separated multi-agent conversations.

Autonomy and workflow structure are orthogonal. Agentless is low-autonomy but high-structure: it localizes files, samples patches, generates reproduction tests, runs regression tests, and ranks candidates [5]. ReAct is higher-autonomy with a comparatively lightweight reason-act loop [1]. FlowBench and Agent Workflow Memory expose task workflows as reusable procedural knowledge [2, 3]. Multi-agent systems add role separation and communication channels, but MAST shows that their failures cluster around system design, inter-agent misalignment, and task verification rather than disappearing through collaboration [8].

We group scaffolds into seven components:

- Planning and decomposition: explicit intermediate steps before action.
- Tool/environment interaction: external observations, API calls, tests, or web/browser feedback.
- Verification: automated checks, executable tests, rubric checks, schema validation, or external judges.
- Reflection/repair: revision after critique or verifier feedback.
- Memory/retrieval: persistent workflows, examples, skills, or document retrieval.
- Role separation: multiple agents with specialized responsibilities.
- Selection/aggregation: ranking, voting, patch selection, or judge panels.

The central design variable is not whether a component exists, but whether it changes the realized graph in a way that improves success faster than it increases model calls, tool calls, latency, and failure surface.

3 Related Work

Reason-act and planning scaffolds. ReAct introduced interleaved reasoning and acting for language models [1]. Tree of Thoughts generalizes linear reasoning into search over intermediate states [9]. These methods motivate planning as a scaffold, but they also introduce a compute-accounting problem: search and repeated attempts must be compared against same-budget sampling baselines.

Reflection and verification. Reflexion and Self-Refine add iterative feedback loops [10], but pure self-correction can fail or degrade reasoning performance without external feedback [11]. CRITIC-style tool-interactive critique motivates our separation between self-reflection and externally grounded verification. The planned experiment conditions therefore separate plan-first from plan-plus-verifier.

Workflow-guided agents. FlowBench provides workflow knowledge in text, code, and flowchart forms across 51 scenarios, 6 domains, 22 roles, 536 sessions, and 4,968 turns [2]. Agent Workflow Memory induces reusable routines and reports WebArena gains, but also depends on evaluator quality and reusable web-navigation structure [3]. Workflow-optimization surveys sharpen the vocabulary: reusable templates, run-specific realized graphs, and execution traces are different objects of evaluation [12]. These works support the positive case for workflow scaffolds while motivating cost and regime controls.

Harness and observation surfaces. Web and software-agent benchmarks make clear that the scaffold is not only the prompt. Mind2Web and VisualWebArena show that HTML filtering, visual grounding, action representation, and benchmark environment design determine what an agent can observe and do [13, 14]. BrowserGym standardizes observation/action spaces and experiment tooling across web-agent benchmarks, turning harness design into an explicit experimental factor [15]. Is Grep All You Need? is the sharpest methodological warning for this draft: retrieval primitive, tool-output delivery path, and provider/native CLI surface can change measured performance enough that they must be logged rather than treated as implementation details [16]. Efficient benchmarking work adds a complementary design rule: use mid-difficulty, rank-preserving task subsets rather than saturated/easy tasks when comparing scaffolds [17].

Concurrent protocol-aligned evaluations now directly test this paper's premise. BenchAgent places single-agent, fixed multi-agent, and evolving multi-agent workflows behind common benchmark-loading, tool-access, answer-contract, accounting, and trajectory-logging interfaces; with GPT-4.1, five of six tested multi-agent systems trailed its matched single-agent anchor, while its external GAIA runtime-workflow result was explicitly treated as a deployed-configuration comparison rather than a mechanism ablation [18]. A larger unified-framework study similarly holds a general-purpose scaffold fixed across benchmarks and reports that scaffold, parser, and environment choices materially shift outcomes over more than 400,000 rollouts [19]. PolyWorkBench reports same-model score spreads of 8--21 points across harnesses on its evaluated pairs [20], and a held-out evaluation of automatic harness evolution finds that it does not consistently beat matched-feedback test-time-scaling baselines in its Terminal-Bench 2.1 setting [21]. These concurrent preprints materially narrow the claim here: this paper does not claim to originate protocol alignment or establish a new empirical ranking. Its contribution is a complementary blocker-regime schema, pilot instrumentation, and an explicit agenda for token/dollar-matched scaffold ablations.

Software agents and fixed workflows. SWE-bench and SWE-agent made software-repair agents a central benchmark and system-design target [22, 4]. Agentless is especially important because it reframes the comparison: a fixed workflow can outperform more autonomous systems in verifier-rich software-repair settings [5]. A newer controlled COBOL-to-Python modernization study isolates execution control itself and reports that deterministic orchestration achieves comparable computational accuracy to LLM-controlled orchestration, improves worst-case robustness, and reduces token consumption by up to 3.5x in structured workflows with explicit validation stages [23]. PIVOT gives the constructive high-structure counterpart: planning failures often arise from plan--execution misalignment, and trajectory refinement uses execution-grounded inspection/evolution/verification rather than plan text alone [24]. SWE-Bench Illusion adds a threat-to-validity warning that popular public benchmarks may reward memorized artifacts rather than generalizable repair reasoning [25].

Agent evaluation and reliability. AgentBench and GAIA motivate broad interactive-agent evaluation [26, 27]. τ -bench introduces final database-state evaluation and pass@k reliability for tool-agent-user interactions [28]. ReliabilityBench generalizes this production-readiness concern into repeated-execution consistency, instruction perturbation, and infrastructure-fault axes; its audited abstract reports that perturbations reduce pass@1 and that ReAct can outperform Reflexion under stress while a cheaper model can match a far more expensive one on reliability [29]. AgentProp-Bench adds a verifier warning: automated tool-agent judges can be poorly calibrated against humans, so verifier reliability should be measured rather than assumed [30]. ClawsBench extends the warning to state-changing productivity agents: a scaffolded agent can achieve task success while still taking unsafe actions, so safe completion should be measured separately from raw success in tool-mutating workflows [31]. ClawSafety adds the adversarial deployment angle: prompt injections through high-trust workspace skill files, emails, and web pages can produce harmful actions, and safety varies with the full model-plus-framework stack rather than the backbone alone [32]. Solver-aided policy checking provides a constructive runtime-gate design for this regime: planned tool calls can be intercepted and checked against formal constraints before execution rather than merely steering the model with policy text [33]. Verifiably safe tool-use and AgentSpec push the same idea into information-flow/tool-sequence specifications and runtime rule enforcement: task-specific agents may need explicit labels, formal rules, enforcement actions, and overhead accounting independent of the model [34, 35]. AI Agents That Matter provides the methodological backbone for this paper: jointly report accuracy and cost, use simple baselines, and identify Pareto-dominated systems [6].

Multi-agent and component interactions. MAST provides a taxonomy of multi-agent failures across 1600+ traces and 7 frameworks [8]. Cross-Component Interference reports that all-in scaffold stacks can be worse than smaller subsets, including a PDF-audited HotpotQA result where tool-use-only outperforms All-In on Llama-3.1-8B [7]. Single-vs-multi-agent token-budget comparisons further warn that multi-agent gains can disappear under equal thinking-token budgets [36]. A direct homogeneous-MAS critique goes further: if agents share the same base model, a single agent executing the workflow through multi-turn conversation can be a stronger and more efficient baseline than a naive single-agent prompt [37]. AgentSlimming gives the constructive counterpart: graph-structured MAS can often be pruned or assigned cheaper models, reporting up to 78.9% token-cost reduction with negligible degradation in its setup [38]. AgentDiet makes the same point at the execution-trace level: multi-turn coding agents can repeatedly pay for stale or redundant trajectory tokens, and inference-time trajectory reduction can cut input-token and total computational cost while preserving benchmark performance in its reported setting [39]. Query-level workflow-generation work similarly argues that per-query workflow synthesis can be redundant and token-expensive compared with reusable task-level workflows [40]. Together these results motivate measuring scaffold interactions, execution-control policy, workflow executor, trajectory/context-retention policy, reusable-template baselines, and minimal/pruned workflow baselines rather than assuming additive gains from larger agent graphs.

4 Method

We compare four conditions:

1. Unstructured: one direct answer.
2. Extra-inference unstructured best-of-2: two direct attempts with verifier selection. This is a causal control for extra generation, not a literally matched-budget comparison to the one-call planning conditions in the present pilots. Future comparisons should set N or token limits from each scaffold’s measured inference budget; broader agent test-time-scaling work makes this family of sampling, revision, verification/merging, and rollout-diversification controls a serious baseline rather than a strawman [41].
3. Plan-first: the model writes a one-line plan before the answer.
4. Plan-plus-check: the model writes a plan and is evaluated by the same verifier. In the deterministic smoke harness this condition can repair after verifier feedback; in the Hermes-delegate canary reported below, repair was not re-run after a

failed verifier check, so it should be interpreted as plan-plus-check rather than a full repair loop.

Each task is represented as JSONL with a prompt, success criterion, verifier, task family, blocker type, verifier reliability/calibration, retrieval mode, tool-delivery path, harness surface, execution-control policy, workflow-executor policy, runtime guardrail/dataflow policy, and safety/state-mutation profile. Verifiers currently include numeric matching, required citation URLs, keyword coverage, strict JSON-field validation, and safe-action JSON checks for state-mutating productivity tasks.

Metrics are success rate, unsafe-action / safe-completion rate when tools mutate persistent state, success per model call, success per tool call, estimated input/output tokens, estimated dollar cost, wall-clock time, failure category, and Pareto-frontier dominance. For production-readiness slices, the protocol should additionally log repeated-run index, instruction perturbation level, injected infrastructure fault, schema drift, and trajectory/context-retention/refinement policy. Runs also log execution control and orchestration policy, because explicit utility-guided orchestration is a separate design variable from the nominal prompt condition [42]. For multi-agent or workflow-graph conditions, runs should record whether the executor is a role-separated graph, a single agent executing the same workflow, or a pruned/minimal graph. For state-mutating tool workflows, runs should separately log `policy_gate`, `runtime_enforcement`, `dataflow_labels`, `tool_sequence_constraints`, and `trust_boundary` so formal guardrails are counted as scaffold components rather than hidden implementation details. For judge-based verifiers, runs should log judge type and any human-calibration evidence. A condition is dominated when another condition has equal or higher success with lower or equal average model/tool/cost burden and at least one strict improvement.

The present implementation has two layers. The deterministic smoke harness validates schema, logging, grouping, and analysis. A separate live adapter is implemented for provider API runs, but the canary reported here uses Hermes-delegate outputs converted into the same `runs.jsonl` schema for analysis.

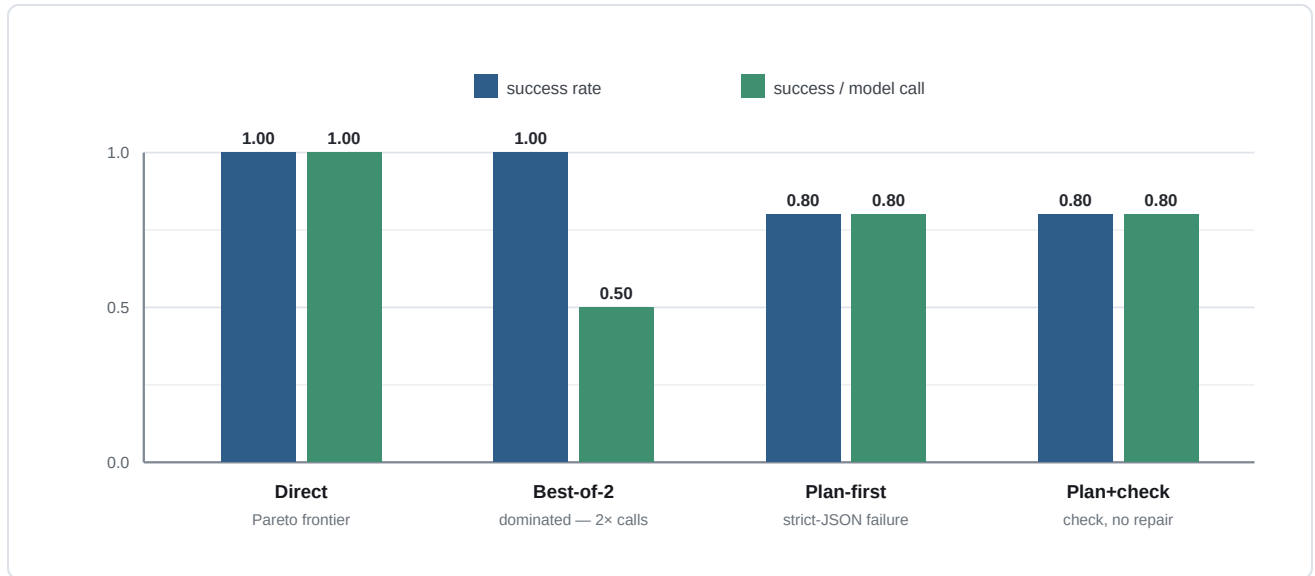


Figure 1. Canary success must be read together with model-call cost. Direct and best-of-2 both solved 5/5 tasks, but best-of-2 spent two model calls per task, halving success per model call. The two planning conditions solved 4/5: visible planning prose preceded the JSON object on a strict-JSON task and the verifier rejected the answer.

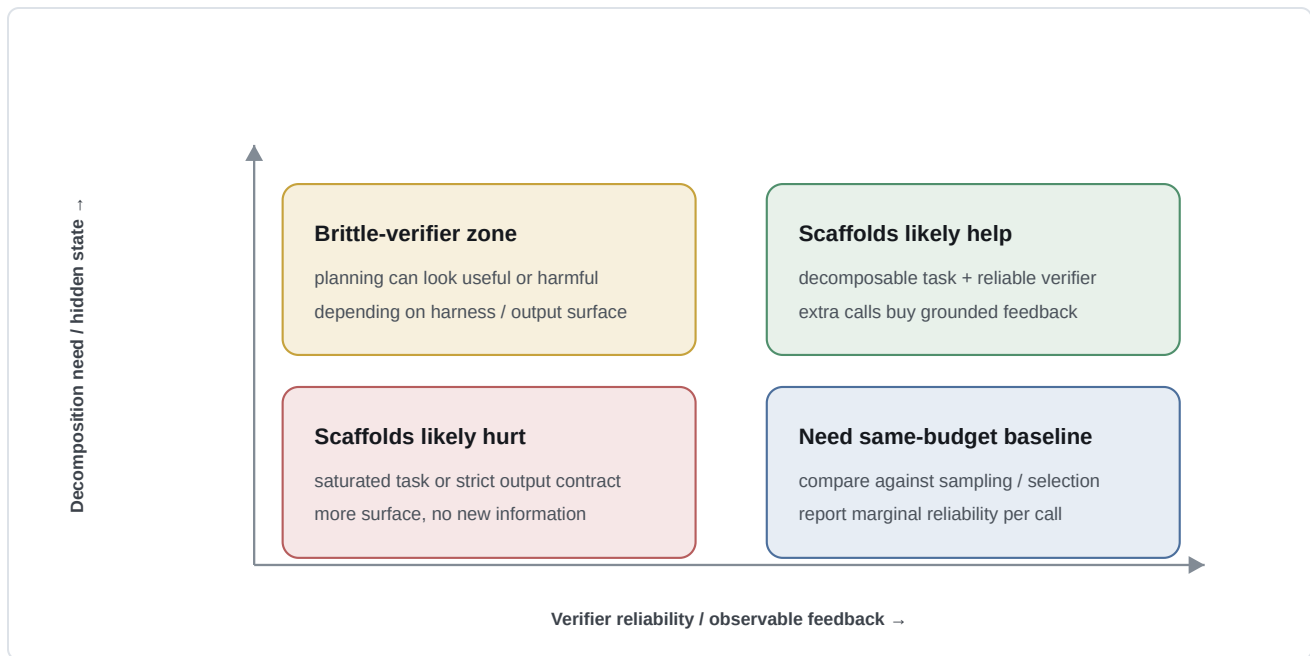


Figure 2. Scaffold value is regime-specific, not monotonic. Workflows are most likely to help when a task is decomposable and the verifier is reliable; most likely to hurt when the task is saturated or the output contract is brittle. The paper’s claim is a measurement claim: evaluate workflow graphs by blocker regime and marginal cost, not by raw success rate alone.

5 Pilot Results

The deterministic smoke harness is not model evidence. It currently completes 18 tasks $\times$ 4 conditions and produces the expected overall, per-family, per-blocker, per-tool-surface, per-execution-control, per-policy-gate/dataflow, and per-safety-profile analysis tables, validating the pipeline.

The Hermes-delegate pilot is also small and should be treated as an illustrative canary, not a publishable benchmark. It produced 20 runs on 5 tasks. Direct unstructured prompting solved all 5 tasks. Plan-first and plan-plus-check solved 4/5. Unstructured best-of-2 solved all 5 but used two logged model calls per task, giving lower success per model call than direct unstructured prompting. The only failures came from the strict JSON task: plan-first and plan-plus-check included a natural-language plan before the JSON object, causing the JSON parser to reject the answer.

This failure is useful. It shows that planning can be actively harmful when the output contract is strict and the scaffold prompt asks for prose that violates the verifier. The result should not be overgeneralized, but it demonstrates why scaffold evaluations need failure categories and harness-surface metadata. A workflow can make the model more explicit while making the system less correct.

Expanded Hermes/Codex 60-task live run

A later expanded live run executed 60 tasks under four conditions, for 240 condition-task runs. All four conditions reached 60/60 success. This is a ceiling-effect result, not evidence that scaffolds do or do not help. It shows that the current expanded task suite is saturated for the Hermes CLI / openai-codex:gpt-5.5 surface. Under logged model/tool-call accounting, plan-plus-check is dominated by plan-first because it adds a verifier/tool call without improving success, and unstructured best-of-2 is dominated by direct unstructured generation because it doubles model calls without improving success. The publishable next step is therefore mid-difficulty task selection rather than further analysis of this saturated run.

Table 1. Expanded 60-task live run (Hermes CLI / openai-codex:gpt-5.5). All conditions reach 60/60; under logged call accounting plan-plus-check and best-of-2 are Pareto-dominated. A ceiling-effect (saturated) result.

Condition	Runs	Success rate	Avg model calls	Avg tool calls	Success/model call	Pareto status
unstructured	60	1.00	1.00	0.00	1.00	frontier
plan_first	60	1.00	1.00	0.00	1.00	frontier
plan_verify	60	1.00	1.00	1.00	1.00	dominated by plan_first
unstructured_best_of_2	60	1.00	2.00	0.00	0.50	dominated by unstructured

6 Limitations

The current live/delegate pilot has only 5 toy tasks and no statistical power. The deterministic smoke suite has expanded to 18 tasks including three synthetic safety/state-mutation checks, but remains synthetic instrumentation, not model evidence. The delegate canary uses Hermes delegate outputs rather than provider-level API calls. Token and cost accounting for the delegate pilot are estimated or unavailable; dollar-cost columns remain `n/a`. Consequently, the reported pilots are normalized only by logged model/tool-call counts, not by tokens, latency, or dollars, and best-of-2 is not budget-matched to the one-call planning conditions. The JSON failure may be a prompt-design artifact, not an inherent flaw in planning. The task set is hand-authored and may over-represent simple tasks where direct prompting is already sufficient. The next empirical step is a preregistered live pilot with mid-difficulty tasks, provider usage logging, explicit cost ceilings, and repeated seeds.

7 Conclusion

The right question is not whether workflows help. The right question is which workflow graph helps which blocker regime at what marginal cost. The current harness makes that question measurable by combining extra-inference controls, verifier-grounded failure categories, regime metadata, and Pareto-frontier reporting; a fully matched-budget study remains future work. Early canary results already show a failure mode that raw scaffold enthusiasm would miss: a plan can break a strict output contract. The next version should replace toy tasks with preregistered live tasks and report scaffold value as reliability per model call, tool call, token, dollar, and verifier regime.

References

- [1] Shunyu Yao et al. (2022). ReAct: Synergizing Reasoning and Acting in Language Models. arXiv:2210.03629v3. <https://arxiv.org/abs/2210.03629v3>
- [2] Ruixuan Xiao et al. (2024). FlowBench: Revisiting and Benchmarking Workflow-Guided Planning for LLM-based Agents. arXiv:2406.14884v1. <https://arxiv.org/abs/2406.14884v1>
- [3] Zora Zhiruo Wang et al. (2024). Agent Workflow Memory. arXiv:2409.07429. <https://arxiv.org/abs/2409.07429>
- [4] John Yang et al. (2024). SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. arXiv:2405.15793v3. <https://arxiv.org/abs/2405.15793v3>
- [5] Chunqiu Steven Xia et al. (2024). Agentless: Demystifying LLM-based Software Engineering Agents. arXiv:2407.01489v1. <https://arxiv.org/abs/2407.01489v1>
- [6] Sayash Kapoor et al. (2024). AI Agents That Matter. arXiv:2407.01502v1. <https://arxiv.org/abs/2407.01502v1>
- [7] Ming Liu (2026). More Is Not Always Better: Cross-Component Interference in LLM Agent Scaffolding. arXiv:2605.05716v1. <https://arxiv.org/abs/2605.05716v1>
- [8] Mert Cemri et al. (2025). Why Do Multi-Agent LLM Systems Fail?. arXiv:2503.13657. <https://arxiv.org/abs/2503.13657>

- [9] Shunyu Yao et al. (2023). Tree of Thoughts: Deliberate Problem Solving with Large Language Models. arXiv:2305.10601v2. <https://arxiv.org/abs/2305.10601v2>
- [10] Noah Shinn et al. (2023). Reflexion: Language Agents with Verbal Reinforcement Learning. arXiv:2303.11366v4. <https://arxiv.org/abs/2303.11366v4>
- [11] Jie Huang et al. (2023). Large Language Models Cannot Self-Correct Reasoning Yet. arXiv:2310.01798. <https://arxiv.org/abs/2310.01798>
- [12] Ling Yue et al. (2026). From Static Templates to Dynamic Runtime Graphs: A Survey of Workflow Optimization for LLM Agents. arXiv:2603.22386v1. <https://arxiv.org/abs/2603.22386v1>
- [13] Xiang Deng et al. (2023). Mind2Web: Towards a Generalist Agent for the Web. arXiv:2306.06070v3. <https://arxiv.org/abs/2306.06070v3>
- [14] Jing Yu Koh et al. (2024). VisualWebArena: Evaluating Multimodal Agents on Realistic Visually Grounded Web Tasks. arXiv:2401.13649v2. <https://arxiv.org/abs/2401.13649v2>
- [15] Thibault Le Sellier De Chezelles et al. (2024). The BrowserGym Ecosystem for Web Agent Research. arXiv:2412.05467v4. <https://arxiv.org/abs/2412.05467v4>
- [16] Sahil Sen et al. (2026). Is Grep All You Need? How Agent Harnesses Reshape Agentic Search. arXiv:2605.15184v1. <https://arxiv.org/abs/2605.15184v1>
- [17] Franck Ndzomga (2026). Efficient Benchmarking of AI Agents. arXiv:2603.23749v1. <https://arxiv.org/abs/2603.23749v1>
- [18] Yuhang Fu et al. (2026). Do More Agents Help? Controlled and Protocol-Aligned Evaluation of LLM Agent Workflows. arXiv:2606.05670v1. <https://arxiv.org/abs/2606.05670v1>
- [19] Pengyu Zhu et al. (2026). A Unified Framework for the Evaluation of LLM Agentic Capabilities. arXiv:2605.27898v2. <https://arxiv.org/abs/2605.27898v2>
- [20] Hongliang Li et al. (2026). PolyWorkBench: Benchmarking Multilingual Long-Horizon LLM Agents. arXiv:2607.06008v2. <https://arxiv.org/abs/2607.06008v2>
- [21] Yike Wang et al. (2026). Rethinking the Evaluation of Harness Evolution for Agents. arXiv:2607.12227v1. <https://arxiv.org/abs/2607.12227v1>
- [22] Carlos E. Jimenez et al. (2023). SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. arXiv:2310.06770. <https://arxiv.org/abs/2310.06770>
- [23] Naing Oo Lwin et al. (2026). Deterministic vs. LLM-Controlled Orchestration for COBOL-to-Python Modernization. arXiv:2605.09894v1. <https://arxiv.org/abs/2605.09894v1>
- [24] Tuo Zhang et al. (2026). PIVOT: Bridging Planning and Execution in LLM Agents via Trajectory Refinement. arXiv:2605.11225v1. <https://arxiv.org/abs/2605.11225v1>
- [25] Shanchao Liang et al. (2025). The SWE-Bench Illusion: When State-of-the-Art LLMs Remember Instead of Reason. arXiv:2506.12286. <https://arxiv.org/abs/2506.12286>
- [26] Xiao Liu et al. (2023). AgentBench: Evaluating LLMs as Agents. arXiv:2308.03688v3. <https://arxiv.org/abs/2308.03688v3>
- [27] Grégoire Mialon et al. (2023). GAIA: a benchmark for General AI Assistants. arXiv:2311.12983. <https://arxiv.org/abs/2311.12983>
- [28] Shunyu Yao et al. (2024). τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. arXiv:2406.12045. <https://arxiv.org/abs/2406.12045>
- [29] Aayush Gupta (2026). ReliabilityBench: Evaluating LLM Agent Reliability Under Production-Like Stress Conditions. arXiv:2601.06112v1. <https://arxiv.org/abs/2601.06112v1>
- [30] Bhaskar Gurram (2026). Evaluating Tool-Using Language Agents: Judge Reliability, Propagation Cascades, and Runtime Mitigation in AgentProp-Bench. arXiv:2604.16706v1. <https://arxiv.org/abs/2604.16706v1>
- [31] Xiangyi Li et al. (2026). ClawsBench: Evaluating Capability and Safety of LLM Productivity Agents in Simulated Workspaces. arXiv:2604.05172v2. <https://arxiv.org/abs/2604.05172v2>
- [32] Wei et al. (2026). ClawSafety: “Safe” LLMs, Unsafe Agents. arXiv:2604.01438v2. <https://arxiv.org/abs/2604.01438v2>
- [33] Cailin Winston et al. (2026). Solver-Aided Verification of Policy Compliance in Tool-Augmented LLM Agents. arXiv:2603.20449v1. <https://arxiv.org/abs/2603.20449v1>
- [34] Aarya Doshi et al. (2026). Towards Verifiably Safe Tool Use for LLM Agents. arXiv:2601.08012v1. <https://arxiv.org/abs/2601.08012v1>
- [35] Haoyu Wang et al. (2025). AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents. arXiv:2503.18666v3. <https://arxiv.org/abs/2503.18666v3>
- [36] Dat Tran et al. (2026). Single-Agent LLMs Outperform Multi-Agent Systems on Multi-Hop Reasoning Under Equal Thinking Token Budgets. arXiv:2604.02460v2. <https://arxiv.org/abs/2604.02460v2>

- [37] Jiawei Xu et al. (2026). Rethinking the Value of Multi-Agent Workflow: A Strong Single Agent Baseline. arXiv:2601.12307v1. <https://arxiv.org/abs/2601.12307v1>
- [38] Yulang Chen et al. (2026). AgentSlimming: Towards Efficient and Cost-Aware Multi-Agent Systems. arXiv:2605.08813v1. <https://arxiv.org/abs/2605.08813v1>
- [39] Yuan-An Xiao et al. (2026). Reducing Cost of LLM Agents with Trajectory Reduction. arXiv:2509.23586v2. <https://arxiv.org/abs/2509.23586v2>
- [40] Zixu Wang et al. (2026). Do We Always Need Query-Level Workflows? Rethinking Agentic Workflow Generation for Multi-Agent Systems. arXiv:2601.11147v1. <https://arxiv.org/abs/2601.11147v1>
- [41] King Zhu et al. (2025). Scaling Test-time Compute for LLM Agents. arXiv:2506.12928v1. <https://arxiv.org/abs/2506.12928v1>
- [42] Boyan Liu et al. (2026). Utility-Guided Agent Orchestration for Efficient LLM Tool Use. arXiv:2603.19896v1. <https://arxiv.org/abs/2603.19896v1>

Typeset from Markdown source; figures and tables rendered in HTML/SVG/CSS. References numbered in order of first citation.